

บันทึกการวัดตัวชี้วัด Buzzly

ผลการวัดตัวชี้วัดทั้งเจ็ดของระบบ Buzzly ตามระเบียบวิธีที่ประกาศไว้ก่อนเริ่มวัด ทุกตัวเลขในหน้านี้อ่านมาจากไฟล์หลักฐานที่เก็บไว้จริง ไม่มีตัวเลขใดถูกประมาณหรือเขียนขึ้นใหม่ และผลที่ไม่ผ่านถูกรายงานไว้เท่ากับผลที่ผ่าน

COMMIT ที่วัด 4c13722	SPEC (PRE-REGISTERED) da02849	ระบบที่วัด buzzly-dev.vercel. app	วันที่ 2026-09-07 ICT
เครื่องมือ Lighthouse 13.4.1 · k6 v2.2.0 · pytest	เครื่อง Ryzen 5 4600H · 12 cores · 15 GB		

KPI-1 ความตรงของข้อมูล เทียบ Meta Ads Manager 0.0000% เกณฑ์ MAPE ≤ 0.5%	KPI-2 อัตราการนำเข้าไฟล์ที่ ถูกต้องสำเร็จ 20 / 20 เกณฑ์ 100%	KPI-3 การดึงไฟล์เสียเข้า Dead-Letter Queue 12 / 12 ครบทั้งสามแกน	KPI-4 คะแนน Lighthouse Performance บนบิลด์ ที่ deploy จริง 65 / 74 / 100 เกณฑ์ ≥ 80 ทุกเส้นทาง
KPI-5 ความหน่วงและอัตราล้ม เหลวที่ผู้ใช้พร้อมกัน 50 ราย p95 = 60 s เกณฑ์ p95 < 2,000 ms	KPI-6 คะแนนความง่ายในการ ใช้งาน (SUS) ยังไม่วัด เกณฑ์ ≥ 68	KPI-7 การประเมินความ ปลอดภัยตาม OWASP Top 10 (2021) 2 / 4 เกณฑ์ผ่านแล้ว 2 ข้อ	

ขั้นที่มาของเกณฑ์ – อ่านตรงนี้ก่อนอ้างตัวเลขใด

เกณฑ์ในงานนี้ไม่ได้มาจากที่เดียวกัน และการอ้างผิดที่มาคือความผิดพลาดที่ร้ายแรงที่สุดที่เกิดขึ้นได้ กับรายงานฉบับนี้ จึงกำกับขั้นที่มาไว้กับทุกตัวชี้วัด

Class 1

ตัวเลขที่อยู่ในข้อเสนอโครงการ §1.3 และผู้วิจัยตรวจสอบยืนยันแล้วเมื่อ 2026-08-14 ได้แก่ Lighthouse ≥ 80 · ผู้ใช้พร้อมกัน 50 ราย · SUS ≥ 68

Class 2

ตัวเลขที่โครงการนี้กำหนดขึ้นเอง ได้แก่ p95 $< 2,000$ ms และอัตราล้มเหลว $< 1\%$ ข้อเสนอเขียนไว้เพียงว่า “รักษาเวลาตอบสนองให้คงที่” โดยไม่ระบุตัวเลข **ห้ามนำเสนอสองค่านี้ว่าเป็นข้อกำหนดจากข้อเสนอ**

ชั้นอื่น

KPI-2 และ KPI-3 มาจาก DLQ sprint brief ส่วนเกณฑ์ของ KPI-7 มาจากคำสั่งอาจารย์ที่ปรึกษาบวกรายการหมวดของ OWASP

ยังไม่ยืนยัน

MAPE $\leq 0.5\%$ ของ KPI-1 ยังไม่ได้ตรวจว่า §1.3 ระบุตัวเลขนี้ไว้หรือไม่ ระหว่างนี้อ้างที่มาเป็น DLQ sprint brief (2026-08-05) ซึ่งเป็นที่ที่รู้แน่ว่ามันถูกเขียนไว้

แถว meta_live ที่ระบบเก็บไว้ ต้องตรงกับไฟล์ export จาก Meta Ads Manager ภายในความคลาดเคลื่อนที่ประกาศไว้ล่วงหน้า

ที่มาของเกณฑ์

ยังไม่ยืนยันชั้น

อ้างอิงจาก DLQ sprint brief (2026-08-05) — ยังไม่ตรวจว่า รุ1.3 ระบุ 0.5% หรือไม่

เครื่องมือ

tests/verify_reconcile.py

หลักฐาน

tests/RESULTS.md · reports/reconciliation-20260813T054823Z.json

ตัวชี้วัด	ค่าที่วัดได้	เกณฑ์
Impressions	0.0000%	≤ 0.5%
Clicks	0.0000%	≤ 0.5%
Spend	0.0000%	≤ 0.5%
CTR	0.0000%	≤ 0.5%
CPC	0.0000%	≤ 0.5%
เซลล์ที่ตรงเป๊ะ	81 / 81	—
แถวที่จับคู่ได้	27 / 27	—
แถวที่มีเฉพาะใน export	0	0

negative control ยิงคสว 3/3 — M1 (แก้ impression 1 ค่า), M2 (ลบ 1 แถว) และ F (ใช้แถว fixture เป็นความจริง) ต่างทำให้ผลกลายเป็น FAIL ตามที่ต้องเป็น การวัดที่ล้มไม่ได้ ไม่ใช่การวัด

การวัดชุดเดียวกันนี้เคยให้ผล **FAIL** ก่อนการปรับนิยาม coverage gate เมื่อ 2026-08-13 เลขคดีไม่เปลี่ยนแม้แต่หลักเดียว ที่เปลี่ยนคือการจัดประเภท 4 แถวที่มีฐานข้อมูลแต่ไม่มีใน export ซึ่งทุกแถวมีค่าเป็นศูนย์ในทุกปริมาณที่นำมาเทียบ **ผลลัพธ์เดิมถูกเก็บไว้คู่กัน ไม่ได้ถูกแทนที่**

ไฟล์ที่ต้องถูกทุกไฟล์ต้องได้ผลลัพธ์ครบตามที่ประกาศไว้ ทั้งสถานะงาน บันทึก DLQ ชุดข้อมูลปลายทาง และการจัดเก็บ

ที่มาของเกณฑ์

ขั้นอื่น

มาจาก DLQ sprint brief ไม่ใช่ตัวเลขในข้อเสนอ

เครื่องมือ

tests/test_ingestion_kpi.py

หลักฐาน

tests/RESULTS.md · tests/evidence/

เกณฑ์ที่วัด	ผล	เกณฑ์
ไฟล์ที่ได้ผลลัพธ์ครบทุกด้าน	20 / 20 (100.0%)	100%
แถวที่รั่วเข้าตารางข้อเท็จจริง	0	0
งานที่สถานะไม่นิ่งภายใน 90 วินาที	0	0

ไฟล์ aux/ ถูกกันออกจากตัวหารทั้งสองตัวชี้วัด เพราะอ่านผ่านแต่ไม่มีตารางปลายทาง และไฟล์ที่ “สำเร็จ” โดยไม่เก็บอะไรเลยไม่ใช้การนำเข้าที่สำเร็จ (ข้อจำกัด L-2)

คอรัปชันถูกตรึงด้วย seed คงที่ และรัน 2 ครั้งได้ผล **เหมือนกันทั้ง 35 fixture**

ไฟล์ที่ผิดรูปแบบทุกไฟล์ต้องถูกปฏิเสธ พร้อมบันทึก DLQ ที่มีรหัสข้อผิดพลาดถูกต้อง และต้องไม่มีแถวใดหลุดเข้าตารางข้อเท็จจริง

ที่มาของเกณฑ์

ชั้นอื่น

มาจาก DLQ sprint brief

เครื่องมือ

tests/test_ingestion_kpi.py

หลักฐาน

tests/RESULTS.md · tests/evidence/

เกณฑ์ที่วัด	ผล	เกณฑ์
มีบันทึก dead-letter	12 / 12 (100.0%)	100%
รหัสข้อผิดพลาดถูกต้อง	12 / 12 (100.0%)	100%
ไม่มีแถวรั่ว + staging ว่าง	12 / 12 (100.0%)	100%

fix_13 (ENCODING_ERROR, UTF-16) ถูกกันออกจากตัวหารในฐานะข้อบกพร่องที่รู้ตัวและบันทึกไว้ จึงนับ 12 ไม่ใช่ 13 บั๊กนี้ถูกตรึงไว้โดยไม่แก้โดยเจตนา เพื่อให้งานวิจัยมีหลักฐานก่อน-หลังเมื่อถึงรอบที่แท้จริง

KPI-4 คะแนน Lighthouse Performance บอนิลด์ที่ deploy จริง B · Performance

FAIL

บิลด์ production ของส่วนที่ถูกค่าใช้ ต้องผ่านเกณฑ์ประสิทธิภาพฝั่งหน้าเว็บบนอุปกรณ์ประเภทที่ระบบถูกสร้างมาเพื่อรองรับ

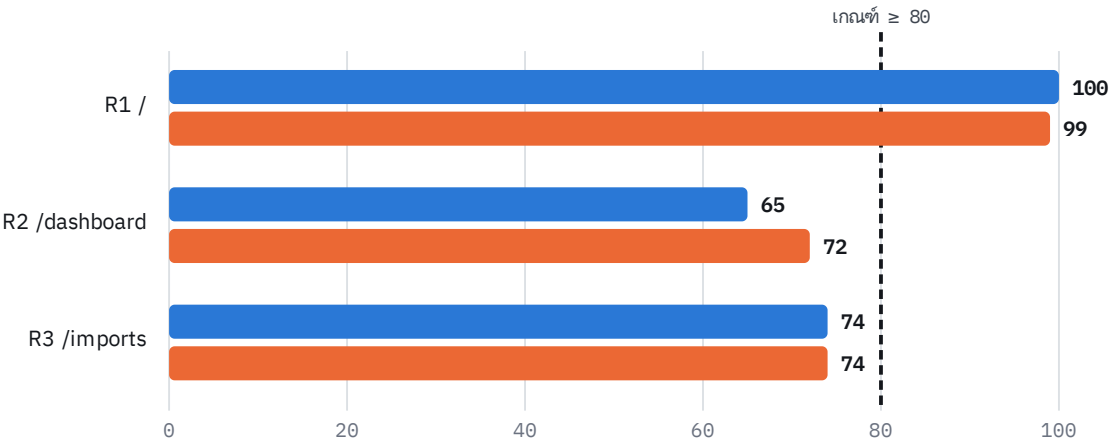
ที่มาของเกณฑ์ Class 1 ≥ 80 อยู่ในข้อเสนอรู1.3 ยืนยันแล้ว 2026-08-14 · ส่วนการเลือก preset ประกาศไว้ในสเปกเอง

เครื่องมือ Lighthouse 13.4.1 · Chrome 145.0.7632.6 · 36 runs (ทั้ง run แรกของทุกช่อง, ใช้ median ของ run 2-6)

หลักฐาน <evidence/kpi4-lighthouse/4c13722/>

เส้นทาง	PRESET	MEDIAN	MIN-MAX	FCP	LCP	TBT	SI
R1 /	desktop	100	100-100	279 ms	284 ms	0 ms	366 ms
R2 /dashboard	desktop	65	65-65	297 ms	14,182 ms	0 ms	5,985 ms
R3 /imports	desktop	74	73-74	286 ms	4,519 ms	0 ms	2,137 ms
R1 /	mobile	99	99-100	875 ms	887 ms	97 ms	875 ms
R2 /dashboard	mobile	72	69-72	887 ms	10,690 ms	187 ms	3,157 ms
R3 /imports	mobile	74	74-75	882 ms	6,878 ms	144 ms	2,543 ms

■ desktop — ตัวตัดสิน ■ mobile — ข้อมูลประกอบ



ภาพที่ 1 คะแนน Lighthouse Performance (0-100) ของสามเส้นทาง วัตบบอนิลด์ที่ deploy จริง commit 4c13722 · ค่าที่แสดงคือ median ของ run ที่ 2-6 หลังทั้ง run แรกของทุกช่อง · เส้นประคือเกณฑ์ ≥ 80 ซึ่ง **ทุกเส้นทางบน desktop ต้องผ่าน** แตกเส้นเดียวคือตกทั้งตัวชี้วัด · แถบ mobile แสดงไว้เป็นข้อมูลประกอบ ไม่มีเกณฑ์ลงทะเบียนไว้และไม่มีผลต่อคำตัดสิน

กฎที่ลงทะเบีย่นไว้ก่อนวัดคือ **ทั้งสามเส้นทางบน desktop ต้อง ≥ 80 ตกเส้นเดียวคือตกทั้งตัวชี้วัด** กฎนี้มีไว้เพื่อไม่ให้ค่าเฉลี่ยของหน้าแรกอู๋หน้า dashboard ไว้ — ถ้าเฉลี่ยกันจะได้ 80 พอดีและ “ผ่าน”

ตัวเลข mobile เป็นข้อมูลประกอบ ไม่ใช่ตัวตัดสิน และไม่มีเกณฑ์ลงทะเบีย่นไว้ — ได้ส่งคำถามถึงอาจารย์ที่ปรึกษาแล้ว เมื่อ 2026-09-07 ว่าจะให้ mobile ตัดสินด้วยหรือไม่ ถ้าให้ ก็คำนวณค่าตัดสินใหม่จากตารางเดียวกันนี้ได้ทันที

การวัดหักล้างสมมติฐานที่สเปกเขียนไว้ล่วงหน้า สเปกระบุผู้ต้องสงสัยไว้ว่าเป็น entry chunk ขนาด 919 kB และการทำ route-level code splitting ผลที่วัดได้ไม่สนับสนุนข้อนั้นเลย บน /dashboard หน้าเว็บพาดครั้งแรกที่ 297 ms, TBT 0 ms, CLS 0.004, root document ตอบใน 60 ms, script bootup 0.4 s, ข้อมูลที่โหลดทั้งหน้า 124 KiB และ Lighthouse **ไม่ขึ้น opportunity audit ที่มี saving แม้แต่รายการเดียว**

เวลาไปอยู่ที่การรอข้อมูล: LCP เท่ากับ Time to Interactive พอดี เพราะไม่มีอะไรใหญ่ขึ้นจอนกว่า query จะกลับมาหน้าเดียวยัง **147 requests โดย 96 รายการเป็น Supabase REST** รายการที่ช้าที่สุดใช้เวลา 8.6 วินาที และหลายรายการเป็น query เดิมซ้ำในโหลดเดียวกัน — workspaces $\times 6$, customer $\times 5$, business_types $\times 5$, industries $\times 5$, ad_accounts $\times 5$

ยังไม่มีการแก้ไขใด ๆ ในตัวแอป สเปกกำหนดให้รอบปรับปรุงเป็นกิจกรรมแยกที่ลงวันที่แยก เพราะ “ก่อน” ที่ถูกแก้แล้วไม่ใช่ “ก่อน” เกณฑ์ไม่ขยับ

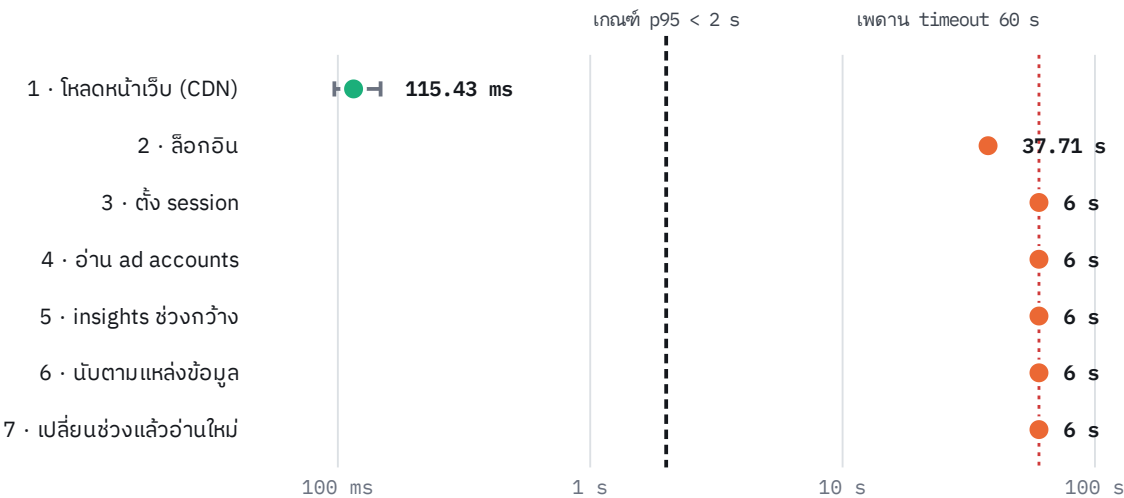
ระบบต้องรักษาเวลาตอบสนองไว้ได้ขณะมีผู้ใช้พร้อมกัน 50 ราย เดินทางการใช้งานจริง 7 ขั้นตอนตั้งแต่โหลดหน้าเว็บ ล็อกอิน จนอ่านข้อมูลโฆษณา

ที่มาของเกณฑ์ ผสม ระดับโหลด 50 VU = Class 1 จากข้อเสนอ §1.3 · แต่ p95 < 2,000 ms และอัตราล้มเหลว < 1% เป็น Class 2 ที่โครงการนี้กำหนดเอง ห้ามอ้างว่าเป็นตัวเลขจากข้อเสนอ

เครื่องมือ k6 v2.2.0 · ramp 0→50 ใน 2 นาที · คง 50 ไว้ 5 นาที · ลดลง 1 นาที · รัน 3 รอบ 16:08 · 16:46 · 17:25 ICT

หลักฐาน evidence/kpi5-k6/4c13722/ · summary.md + run-1..3/

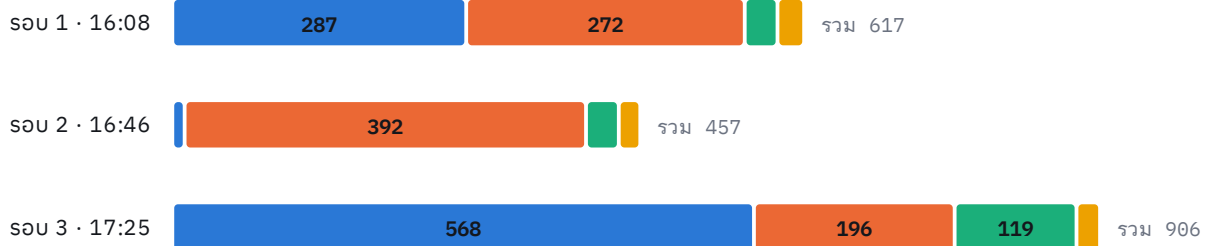
สิ่งที่วัด	MEDIAN (3 รอบ)	MIN	MAX	เกณฑ์
http_req_duration p95	60.00 s	60.00 s	60.00 s	< 2,000 ms
อัตราล้มเหลว http_req_failed	60.50%	50.94%	65.13%	< 1%
checks ที่ผ่าน	39.80%	34.86%	49.05%	—
รอบการใช้งานที่จบครบ	133	110	233	—
ขั้น 1 · โหลดหน้าเว็บจาก CDN	115.43 ms	96.89 ms	147.69 ms	—
ขั้น 2 · ล็อกอิน	37.71 s	37.35 s	41.08 s	—
ขั้น 3 · ตั้ง session	60.00 s	60.00 s	60.00 s	—
ขั้น 4 · อ่าน ad accounts	60.00 s	60.00 s	60.00 s	—
ขั้น 5 · อ่าน insights ช่วงกว้าง	60.00 s	60.00 s	60.00 s	—
ขั้น 6 · นับตามแหล่งข้อมูล	60.00 s	60.00 s	60.00 s	—
ขั้น 7 · เปลี่ยนช่วงแล้วอ่านใหม่	60.00 s	60.00 s	60.00 s	—



■ ชั้นที่ไม่แตะฐานข้อมูล ■ ชั้นที่แตะฐานข้อมูล

ภาพที่ 2 เวลาตอบสนองที่เปอร์เซ็นไทล์ 95 ของแต่ละชั้นในเส้นทางการใช้งาน ขณะมีผู้ใช้พร้อมกัน 50 ราย · จุดคือ median ของสามรอบ เส้นคาดคือช่วงต่ำสุด-สูงสุด (ชั้นที่ 3-7 ไม่มีช่วงเพราะชนเพดานเท่ากันทั้งสามรอบ) · **แกนนอนเป็นลอการิทึม** เพราะค่าครอบคลุมตั้งแต่หลักร้อยมิลลิวินาทีถึงหกหมื่น · ชั้นที่ 1 เป็นชั้นเดียว ที่ไม่แตะฐานข้อมูลและเป็นชั้นเดียวที่ไม่เคยถูกรุด

■ HTTP 500 ■ ไม่ได้รับคำตอบ ■ 429 ■ 504



ภาพที่ 3 จำนวนคำขอที่ล้มเหลว แยกตามรหัสตอบกลับ ในแต่ละรอบการทดสอบ · ความยาวแท่งคือจำนวนครั้งจริง ไม่ใช่สัดส่วน จึงเทียบทั้งปริมาณและองค์ประกอบได้พร้อมกัน · **รูปแบบไม่คงที่ระหว่างรอบ** รอบ 2 เป็นการหมดเวลารอเกือบทั้งหมดโดยมี HTTP 500 เพียง 12 ครั้ง ขณะที่รอบ 1 และ 3 มี 500 เป็นหลัก ระบบจึงอ้อมตัวถูกรอบแต่ไม่ได้อ้อมตัว ผ่านทางเดียวกัน · รหัส 429 เป็นส่วนน้อยถูกรอบ การถูกจำกัดอัตราจึงไม่ใช่คอขวดหลัก

ค่า p95 ที่ 60.00 วินาทีคือ**เพดานเวลารอของคำขอ** พูดย่ออีกอย่างคือ ณ เปอร์เซ็นไทล์ที่ 95 คำขอนั้น**ไม่ได้เสร็จเลย** ไม่ใช่ว่าเสร็จช้า และตัวเลขนี้เท่ากันทั้งสามรอบ

ที่ผู้ใช้ 1 ราย ระบบเดียวกันนี้ให้ p95 = 1.11 วินาที และผ่าน checks 10/10 วัดก่อนรอบแรกไม่กี่นาที ความล้มเหลวจึงเป็นเรื่องของการรับโหลด ไม่ใช่เส้นทางการใช้งานผิด

ชั้นที่ 1 คือชั้นเดียวที่ไม่เคยถูกรุด — มันเป็นชั้นเดียวที่ไม่แตะฐานข้อมูล คือดึงไฟล์สแตติกจาก CDN ของ Vercel ซึ่งอยู่ราว 100 มิลลิวินาทีตลอดการทดสอบ ขณะที่ทุกชั้นที่พึ่งฐานข้อมูลชนเพดาน 60 วินาทีพร้อมกัน **จุดอ้อมตัวจึงอยู่ที่ชั้นข้อมูล ไม่ใช่ที่การส่งไฟล์**

รูปแบบความล้มเหลวไม่คงที่ระหว่างรอบ ซึ่งเป็นผลที่ต้องรายงานตามจริง รอบ 1 เป็น HTTP 500 จำนวน 287 กับหมดเวลารอ 272 · รอบ 2 กลับกลายเป็นหมดเวลารอ 392 โดยมี 500 เพียง 12 · รอบ 3 เป็น 500 ถึง 568 ระบบจึงอ้อมตัวถูกรอบ **แต่ไม่ได้อ้อมตัวผ่านประตูเดียวกัน** การวัดชุดนี้จึงยังไม่ระบุกลไกเดียวที่แน่ชัด และการฟันธงว่าเป็นกลไกใดจะเกินกว่าที่ข้อมูลรองรับ สิ่งที่ยืนยันได้คือ **การถูกจำกัดอัตรา (429) เป็นส่วนน้อยในทุกรอบ ไม่ใช่คอขวดหลัก**

ผลนี้มาจากสาเหตุเดียวกับ KPI-4: หนึ่งการโหลด dashboard ยิง 96 คำขอไปยัง Supabase ที่ผู้ใช้พร้อมกัน 50 ราย จึงเป็นระดับหลายพันคำขอพร้อมกันบนฐานข้อมูลระดับ free tier **ตัวชี้วัดสองตัวที่ไม่ผ่าน คือข้อบกพร่องเดียวที่มองจากคนละด้าน**

ข้อจำกัดที่ต้องระบุคู่กับตัวเลขเสมอ: ผู้ใช้ทดสอบ 10 บัญชีอยู่ใน workspace เดียวกัน จึงเป็น “สิบตัวตน หนึ่งในผู้เช่า” — ได้การประเมิน RLS สิบเส้นทางและ JWT สิบใบจริง แต่ทั้งหมดอ่านแถวชุดเดียวกัน ห้าสิบร้านคำจริงจะถือข้อมูลคนละชุดและกระทบดัชนีคาดเดายากกว่านี้มาก

ยังไม่มีทางเลือกใด ๆ เพื่อให้ตัวเลขดีขึ้น สามรอบนี้คือ “ก่อน” ตามที่สเปกกำหนดให้รอบปรับปรุงเป็นกิจกรรมแยกที่ลงวันที่แยก

ผู้ใช้ที่เคยยิงโฆษณาหรืออ่านตัวเลขผลโฆษณาด้วยตนเองในรอบ 12 เดือน ต้องให้คะแนน System Usability Scale หลังทำการกิจจริงจบ

ที่มาของเกณฑ์

Class 1 SUS ≥ 68 อยู่ในข้อเสนอ §1.3 ยืนยันแล้ว 2026-08-14

เครื่องมือ

SUS (Brooke 1996) 10 ข้อ + ตัววัดภารกิจ

หลักฐาน

evidence/kpi6-sus/protocol.md · form-th.md · evidence/approvals/

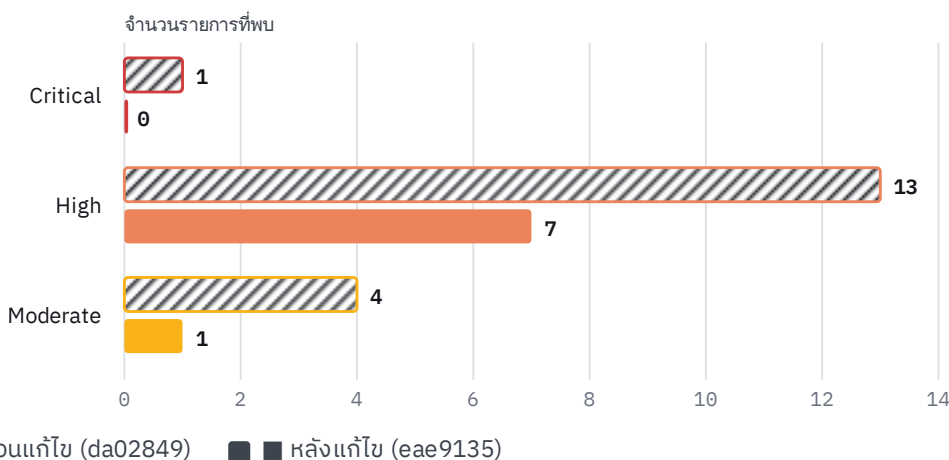
สิ่งที่เตรียมไว้แล้ว	สถานะ
โปรโตคอลการทดสอบ (160 บรรทัด)	พร้อม
แบบสอบถามไทย/อังกฤษ	พร้อม
บันทึกอนุมัติวิธีคัดผู้เข้าร่วม	อนุมัติด้วยวาจา · รอเป็นลายลักษณ์อักษร
จำนวนผู้เข้าร่วมที่เก็บได้	0 (เป้าหมาย 8–12 ขั้นต่ำ 5)

- ตัวชี้วัดนี้ติดตามการหาผู้เข้าร่วม ไม่ได้ติดตามที่ระบบหรือเครื่องมือ
- แผนสำรวจ (Nielsen heuristic evaluation หรือ cognitive walkthrough) ถูกขออนุมัติล่วงหน้าไว้แล้ว จะใช้ก็ต่อเมื่อหาผู้เข้าร่วมได้ไม่ถึง 5 ราย และจะบันทึกเป็นการเบี่ยงเบนที่ลงวันที่กำกับ

ต้องตอบครบทั้ง 10 หมวดของ OWASP Top 10 ว่าใช้กับระบบนี้อย่างไร และต้องไม่เหลือช่องโหว่ระดับ High หรือ Critical

ที่มาของเกณฑ์ ชั้นอื่น เกณฑ์มาจากคำสั่งอาจารย์ที่ปรึกษา บวกกับรายการหมวดของ OWASP เอง
เครื่องมือ ตารางความเกี่ยวข้อง + OWASP ZAP baseline + npm audit
หลักฐาน evidence/kpi7-security/{da02849, eae9135, fc86210}/

เกณฑ์	สถานะ	รายละเอียด
1 · ตอบครบ 10 หมวด	ผ่าน	10 / 10 หมวดมีคำตอบพร้อมเหตุผล
2 · OWASP ZAP baseline	ยังไม่รัน	เพิ่งเลิกติด deploy วันนี้
3 · ไม่มีช่องโหว่ High/Critical	ผ่านหลังแก้ไข	Critical 1 → 0
4 · รันซ้ำได้ผลตรงกัน	ตรงเดียว	npm audit ตรงกัน 2 รอบ · ZAP ยังไม่รัน



ภาพที่ 4 จำนวนช่องโหว่ที่ npm audit รายงาน แยกตามระดับความรุนแรง ก่อนและหลังรอบแก้ไขที่ลงวันที่แยก · แ่งลายทแยงคือก่อนแก้ แ่งทึบคือหลังแก้ · **Critical ลดจาก 1 เหลือ 0** ซึ่งเป็นเงื่อนไขของเกณฑ์ข้อ 3 · จำนวนแพ็กเกจใน production tree ลดจาก 365 เหลือ 188 แต่ไม่นำมาวาดในภาพเดียวกัน เพราะเป็นคนละหน่วยและคนละมาตราส่วน

เกณฑ์ข้อ 3 **เคย FAIL และถูกเก็บผล FAIL นั้นไว้ไม่ลบ** การแก้ไขทำเป็นรอบที่ลงวันที่แยก แล้วรายงานคู่กัน: ก่อนแก้มี Critical 1 · High 13 · Moderate 4 รวม 18 รายการ บน 365 แพ็กเกจ หลังแก้เหลือ **Critical 0 · High 7 · Moderate 1 รวม 8 รายการ บน 188 แพ็กเกจ** โดยไม่มีเกณฑ์ใดขยับ

ผลของแต่ละการแก้ถูกวัดทีละขั้นเพื่อให้ระบุที่มาได้: อัปเดต jspdf 4.2.0 → 4.2.1 ทำให้ Critical เหลือศูนย์ · ย้าย tailwindcss-animate ไป devDependencies ตัดไป 7 รายการ · ถอด @react-three/* และ three ออก เหลือ 8 รายการสุดท้าย

การวัดซ้ำ

ทุกตัวเลขในหน้านี้วัดซ้ำได้จากที่เก็บโค้ดเดียวกัน คำสั่งด้านล่างคือคำสั่งที่ใช้จริง

```
python3 -m pytest tests/test_ingestion_kpi.py -v          # KPI-2 · KPI-3 (~21 นาที)
python3 tests/verify_reconcile.py                        # KPI-1
LIGHTHOUSE_DIR=<dir> node scripts/kpi4-lighthouse.mjs \
  --site https://buzzly-dev.vercel.app                    # KPI-4 (36 runs)
K6_USER_PASSWORD=... SITE_URL=https://buzzly-dev.vercel.app \
  k6 run k6/kpi5-dashboard-journey.js                     # KPI-5 (8 นาทีต่อรอบ)
```

Lighthouse ถูกติดตั้งไว้บนอกที่เก็บโค้ดโดยเจตนา เพราะเกณฑ์ข้อ 4 ของ KPI-7 นับผลจากnpm audit บนต้นไม้ dependency ของโปรเจกต์ การเอาเครื่องมือวัดไปใส่ในนั้น จะไปขยับตัวเลขที่ตัวชี้วัดอีกตัวรายงานอยู่